📘 Django Model Forms & Meta Class — Notes

🧩 1️⃣ What are Django Forms?

Django **Forms** help to handle **user input** easily and safely.

They are used to:

- Display HTML input fields automatically.
- Validate user input.
- Save data to the database.
- Protect from invalid or unsafe data.

There are **two main types**:

1. **Normal Forms** — written manually.
2. **Model Forms** — created automatically from models.

🧱 2️⃣ What are Model Forms?

Model Forms are special Django forms that are **directly connected to a model**.

➡️ It automatically creates form fields from model fields.
➡️ It reduces boilerplate code.

Model Forms in Django are special forms that are **automatically created from models**.
They allow you to **create, update, and validate** model data directly from HTML forms **without writing repetitive code**.

STEP 1: Add Category Model in `models.py`

```
class Category(models.Model):
    category_name=models.CharField(max_length=200)
    cat_description=models.TextField()
```

```
class Book(models.Model):

title = models.CharField(max_length=200)

author = models.CharField(max_length=100)

category = models.ForeignKey(Category,on_delete=models.SET_NULL, null=True, blank=True)

price = models.DecimalField(max_digits=6, decimal_places=2)

description = models.TextField()
```

Step 2: Create a file `forms.py` inside your app

If you don't already have one, create `forms.py` in the same folder as `models.py`.

Then add:

```
from django import forms
from .models import Category

class CategoryForm(forms.ModelForm):
    class Meta:
        model = Category            # The model this form is linked to
        fields = ['category_name', 'cat_description']  # The model
fields to include in the form
```

What is the Meta Class in Django ModelForms?

In Django, every **ModelForm** has an **inner class** called `Meta`.
It is **not a separate model** or **Python meta class —** it's just a **nested class** inside your form that **tells Django how the form should behave**.

What Does the Meta Class Do?

The `Meta` class provides **metadata** — i.e., information *about* the form — such as:

Option	Purpose
`model`	Tells Django which model this form is based on
`fields`	List or tuple of fields to include in the form
`exclude`	List of fields to exclude (alternative to `fields`)
`labels`	Customize field labels
`help_texts`	Add help text for each field
`widgets`	Customize HTML input types for form fields

Example with All Meta Options

```
class CategoryForm(forms.ModelForm):
    class Meta:
        model = Category
        fields = ['category_name', 'cat_description']

        labels = {
            'category_name': 'Category Name',
            'cat_description': 'Description',
        }

        help_texts = {
            'category_name': 'Enter a short and unique name for the
category.',
        }

        widgets = {
            'category_name': forms.TextInput(attrs={'class':
'form-control', 'placeholder': 'Enter category name'}),
```

```
            'cat_description': forms.Textarea(attrs={'class':
'form-control', 'rows': 3}),
        }
```

🗒️ In Simple Terms:

Think of the `Meta` class as a **settings box** 📦 inside your form that says:

> “Hey Django, this form is for this model, and I want these fields shown in this way.”

🧩 Analogy:

If a Django form is a “blueprint”,
then the **`Meta` class** is the **instruction sheet** that tells Django how to build that blueprint from your model.

Step 3: Use it in your `views.py`

```
from django.shortcuts import render, redirect
from .forms import CategoryForm

def add_category(request):
    if request.method == 'POST':
        form = CategoryForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('category_list')  # or any page you want
    else:
        form = CategoryForm()
    return render(request, 'add_category.html', {'form': form})
```

Step 4: Create `add_category.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>Add Category</title>
</head>
<body>
    <h2>Add Category</h2>
    <form method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Save</button>
    </form>
</body>
</html>
```

Step 5: Display all categories

In `views.py`:

```
from .models import Category

def category_list(request):
    categories = Category.objects.all()
    return render(request, 'category_list.html', {'categories':
categories})
```

In `category_list.html`:

```
<h2>All Categories</h2>
<ul>
{% for cat in categories %}
  <li>{{ cat.category_name }} - {{ cat.cat_description }}</li>
{% endfor %}
</ul>
```